# STL analysis of Waterville monitoring data – Part II: STL Functions

Song Qian

## Introduction

STL analysis of long-term nutrient monitoring data from Heidelberg University. Data retrieved from National Center for Water Quality Research: https://ncwqr.org/ and processed in Part I.

## Front matters

Setting up proper file paths, defining some useful functions, and loading necessary packages

## STL Analysis

Here we use two functions to conduct STL analysis using monthly averages versus using daily values. When using monthly averages, we emphasize the seasonal (month) patterns, whereas daily data STL emphasizes the long-term trend. The two functions extract the fitted STL object for various components (seasonal, trend, and residuals) to construct different plots.

```
my.stl.monthly <- function(data = carbon.dioxide,
                                ylab = list(label="Carbon Dioxide (ppm)", cex=1),
                                xlab = list(label="Year", cex=1),
                                axes.cex=0.75,
                               aspect = "xy",
                               s.win=25, s.deg=1, t.win=125, t.deg=1,
                                Ylim=NULL){
  the.fit <- stl(data, s.window = s.win, s.degree = s.deg,
                  t.window = t.win, t.degree = t.deg)
  sfit <- the.fit$time.series[,1]
  tfit <- the.fit$time.series[,2]
```

```
  fit.time <- time(data)
  car.subseries <- factor(cycle(data), label = month.abb)
  obj1.df <- data.frame(y=sfit, x=fit.time, cat=car.subseries)

  subs=car.subseries %in% levels(car.subseries)[3:7]

  obj1_5 <- xyplot(sfit[subs] ~ fit.time[subs] | car.subseries[subs],
                   layout = c(5,1), panel = function(x, y){
                       panel.xyplot(x, y, type = "l")
                       panel.abline(h = mean(y))},
                   ##        aspect = aspect,
                     xlab = xlab,
                     ylab = ylab,
                   scale=list(cex=axes.cex)
                     )
  obj1 <- xyplot(sfit ~ fit.time| car.subseries, layout = c(12,1),
                   panel = function(x, y){
                     panel.xyplot(x, y, type = "l")
                     panel.abline(h = mean(y))},
                   ##        aspect = aspect,
                   xlab = xlab,
                   ylab = ylab,
                 scale=list(cex=axes.cex)
  )

  obj2 <- xyplot(sfit~fit.time|equal.count(fit.time, 7, overlap=0.1),
                   panel = function(x, y) {
                     panel.grid(v = 0)
                     panel.xyplot(x, y, type="l")
                   },
                   aspect = aspect,
                   strip = F,
                   scale = list(x = "free", cex=axes.cex),
                   layout = c(1,7),
                     xlab = xlab,
                     ylab = ylab)

n <- length(data)
the.fit.trend<-the.fit$time.series[,2]-mean(the.fit$time.series[,2])
fit.components <- c(the.fit.trend, the.fit$time.series[,1],
                        the.fit$time.series[,3])
fit.time <- rep(time(data), 3)
```

```
fit.names <- ordered(rep(c("Trend", "Seasonality", "Residuals"),
                           c(n, n, n)),
                     c("Trend", "Seasonality", "Residuals"))

obj3.df <- data.frame(x=fit.time, y=fit.components,
                      series=fit.names)
obj3 <- xyplot(fit.components ~ fit.time | fit.names,
               panel = function(x, y){
                   panel.grid(h = 5)
                   panel.xyplot(x, y, type = "l")
                 },
                 aspect=0.75, layout = c(3, 1),
                 ylim = c(-1, 1) * max(abs(fit.components)),
                 xlab = xlab,
               ylab = ylab,
               scale=list(cex=axes.cex)
)
 obj4 <- xyplot(data, panel=function(x,y){
                      panel.grid(h=5)
                      panel.xyplot(x,y,type="l")
                    },
               aspect=0.75,
               xlab = xlab,
               ylab = ylab,
               scale=list(cex=axes.cex)
 )

 if (!is.null(Ylim))
  obj5 <- xyplot(tfit~fit.time,
                 panel = function(x, y, a=(t.win/12)/2){
                 panel.grid(h = 5)
                 panel.xyplot(x, y, type = "l")
                 panel.abline(v= c(min(x)+a, max(x)-a), col=gray(0.25))
               }, ylim=Ylim,
               xlab = xlab,
               ylab = ylab,
               scale=list(cex=axes.cex))
 else
     obj5 <- xyplot(tfit~fit.time,
                 panel = function(x, y, a=(t.win/12)/2){
                 panel.grid(h = 5)
                 panel.xyplot(x, y, type = "l")
```

```
                panel.abline(v= c(min(x)+a, max(x)-a), col=gray(0.25))
              },
              xlab = xlab,
              ylab = ylab,
              scale= list(cex=axes.cex))

 return(list(dataplot=obj4, stl_comp=obj3, season12=obj1,
             season5=obj1_5, seasonosc=obj2, trend=obj5))
}

my.stl.daily <- function(data = carbon.dioxide,
                                ylab = list(label="Carbon Dioxide (ppm)", cex=1),
                                xlab = list(label="Year", cex=1),
                                axes.cex=0.75, daily.main=NULL,
                              aspect="xy", s.win=725, s.deg=1,
                              t.win=365, t.deg=1){
the.fit <- stl(data, s.window = s.win, s.degree = s.deg,
                   t.window = t.win, t.degree = t.deg)
sfit <- the.fit$time.series[,1]
tfit <- the.fit$time.series[,2]
fit.time <- time(data)
fit.month <- factor(substring(as.yearmon(time(data)), 1,3), label=month.abb)
ylim <- c(-1,1)*max(abs(sfit))
obj1 <- xyplot(sfit ~ fit.time | equal.count(fit.time, 7, overlap=0.1),
                   panel = function(x, y) {
                   panel.grid(v = 0)
                   panel.xyplot(x, y, type="l")
                 },
                 aspect = aspect,
                 strip = F,
                 ylim = ylim,
                 scale = list(x = "free", cex=axes.cex),
                 layout = c(1, 7),
               xlab = xlab,
               ylab = ylab)
obj2 <- xyplot(sfit ~ fit.time | fit.month, layout = c(12, 1),
                   panel = function(x, y){
                   panel.xyplot(x, y, type="l")
                   panel.abline(h = mean(y))},
           ##aspect = 1/5,
                 xlab = xlab,
              ylab = ylab,
```

```
                scale=list(cex=axes.cex)
)
##plot(the.fit, main=daily.main)
return(list(obj1, obj2, the.fit))
}
```

### Running STL

Using the following functions to generate figures

```
## All figures for online materials
## fitting STL and generating plots for supporting materials
STL_Mplots_online <- function(Var = "TP", TSm = TP_ts_monthly,
                      TSd = TP_ts_daily,
                          aspect="xy",
                          ylab=list(label="log TP (mg/L)", cex=0.8),
                      xlab=list(label="Year", cex=0.8),
                      axes.cex = 0.7,
                        ms.win=55, ss.deg=1, mt.win=135, tt.deg=1,
                        ds.win=705, dt.win=3650,
                        Log=T, precip=F, ...)
{
  ## Monthly plots
  if (Log) {
    month_ts <- log(TSm)
    if (!precip) daily_ts <- log(TSd)
  } else {
    month_ts <- TSm
    daily_ts <- TSd
  }
  M_objs <- my.stl.monthly (data = month_ts, xlab=xlab, ylab = ylab, axes.cex=axes.cex,
                            aspect = aspect,
                            s.win=ms.win, s.deg=ss.deg,
                            t.win=mt.win, t.deg=tt.deg, ...)
return(M_objs)
}

STL_Dplots_online <- function(Var = "TP", TSm = TP_ts_monthly,
                      TSd = TP_ts_daily,
                          aspect="xy",
                          ylab=list(label="log TP (mg/L)", cex=0.8),
```

```
                      xlab=list(label="Year", cex=0.8),
                      axes.cex = 0.7,
                        ms.win=55, ss.deg=1, mt.win=135, tt.deg=1,
                        ds.win=705, dt.win=3650, files=T,
                        Log=T, ...)
{
  ## Monthly plots
  if (Log) {
    month_ts <- log(TSm)
    if (!precip) daily_ts <- log(TSd)
  } else {
    month_ts <- TSm
    daily_ts <- TSd
  }
  D_obj <- my.stl.daily (data = daily_ts, xlab=xlab, ylab = ylab, axes.cex=axes.cex,
                                  aspect = 1/5, s.win=ds.win, s.deg=1,
                                  t.win=dt.win, t.deg=1)
return(D_obj)
}

save(my.stl.monthly, my.stl.daily, STL_Mplots_online, STL_Dplots_online,
     file=paste(dataDIR, "stlFunctions.RData", sep="/"))
```

**End of Part II**